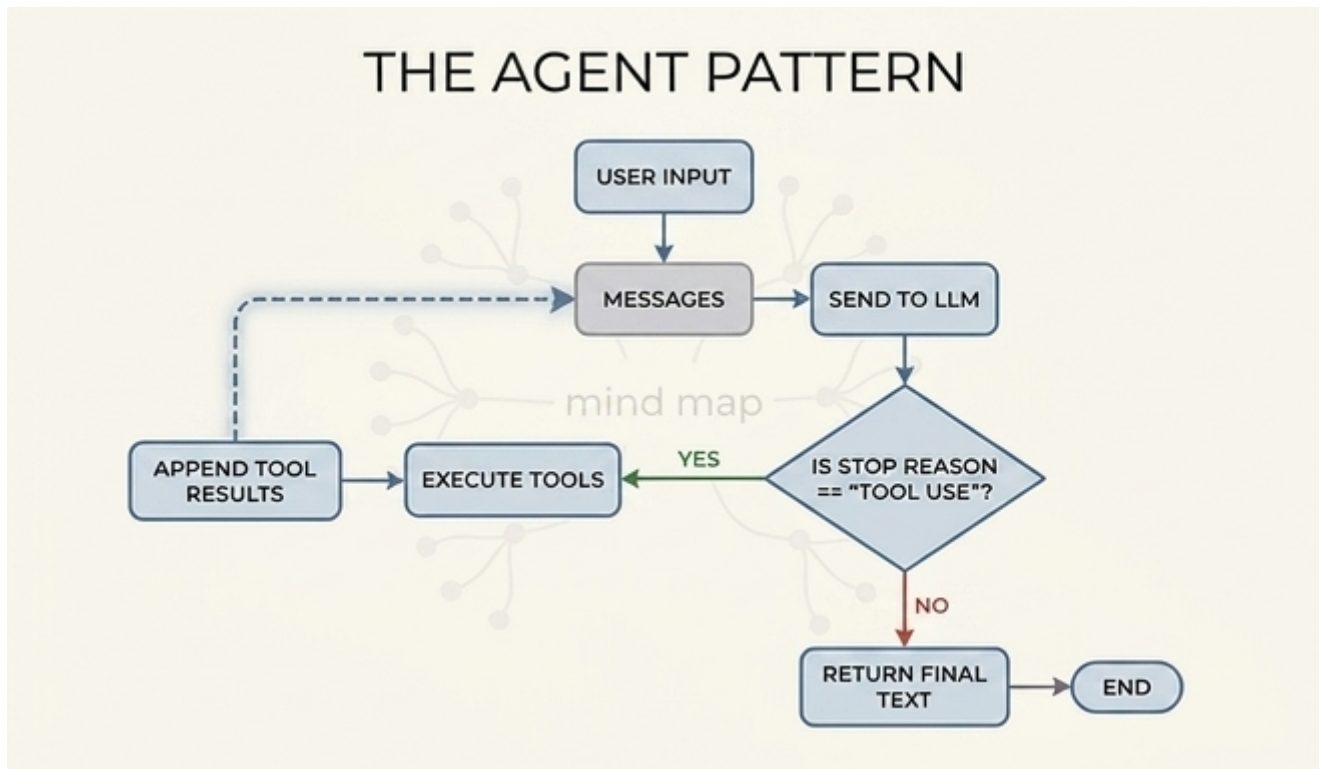


Agent 系统架构

最小心智模型



- 为什么要使用工具？

LLM很强大，但是仍旧是一个语言预测的机器，没有与外界交互的方式。有了工具，LLM就可以感知外部世界、行动、验证和迭代。能思考 + 能做事。

- 第一个takeaway

正常情况下，不考虑任何的可能的错误，模型的停止只有一个情况，就是输出了纯文本，没有 `tool_call`。此时才会输出最终的答案。否则就是一个反复调用工具的过程。

Turn

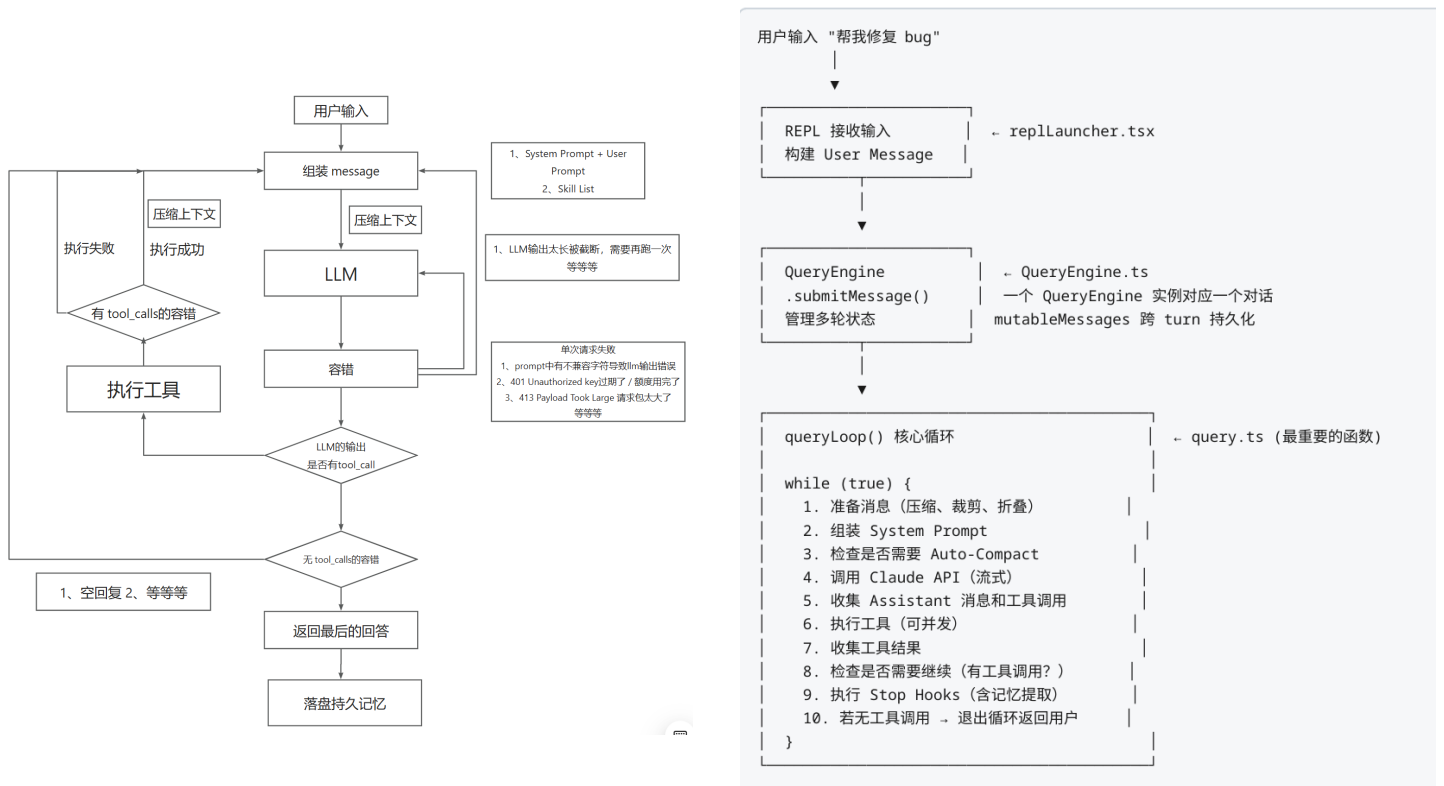
Tool call1: 写文件工具

Tool call2: bash工具

No tool call: 文本形式的最后答案



深入一次turn的全过程 —— 核心运行流程



2. 总流程（一眼图）

```
构造/配置 AIAgent (主要在 __init__, 此处略)
|
▼
run_conversation(user_message, history, ...)    ← 外部 (CLI/Gateway) 几乎都调这个
|
├─ 回合前准备 (会话、预算、中断、连接、system prompt、预压缩...)
|
└─ while 主循环 (迭代预算 + max_iterations)
    |
    ├─ 组 api_messages (含仅用于 API 的注入: 记忆 prefetch、插件等)
    ├─ _build_system_prompt (首轮/缓存) 已在循环外准备好 active_system_prompt
    |
    └─ while 内层「单次 API 重试」
        _build_api_kwargs → _interruptible_streaming_api_call
                           或 _interruptible_api_call
        |
        ├─ 有 tool_calls → _build_assistant_message → _execute_tool_calls
        |                   → 可能 _compress_context → continue 大循环
        |
        └─ 无 tool_calls → 视为「可结束的自然语言」
                           (中间大量 empty/截断/codex 等恢复分支)
                           → break 大循环
    |
    ├─ 若仍无 final: _handle_max_iterations (用尽迭代时的收尾摘要)
    ├─ _save_trajectory、_cleanup_task_resources、_persist_session
    └─ 组装 result dict, 可选 memory / 插件 post_llm_call 等
```

从最小心智模型到初级版 Agent Harness

- 概念区分：（以使用cursor为例）



多个对话框 (session)

Turn1

Turn2

单个session的多个turn, 对话

Tool call1: 写文件工具

Tool call2: bash工具

No tool call: 文本形式的最后答案

Session: 一条持久化的对话线程, 多轮用户消息共享一条线程

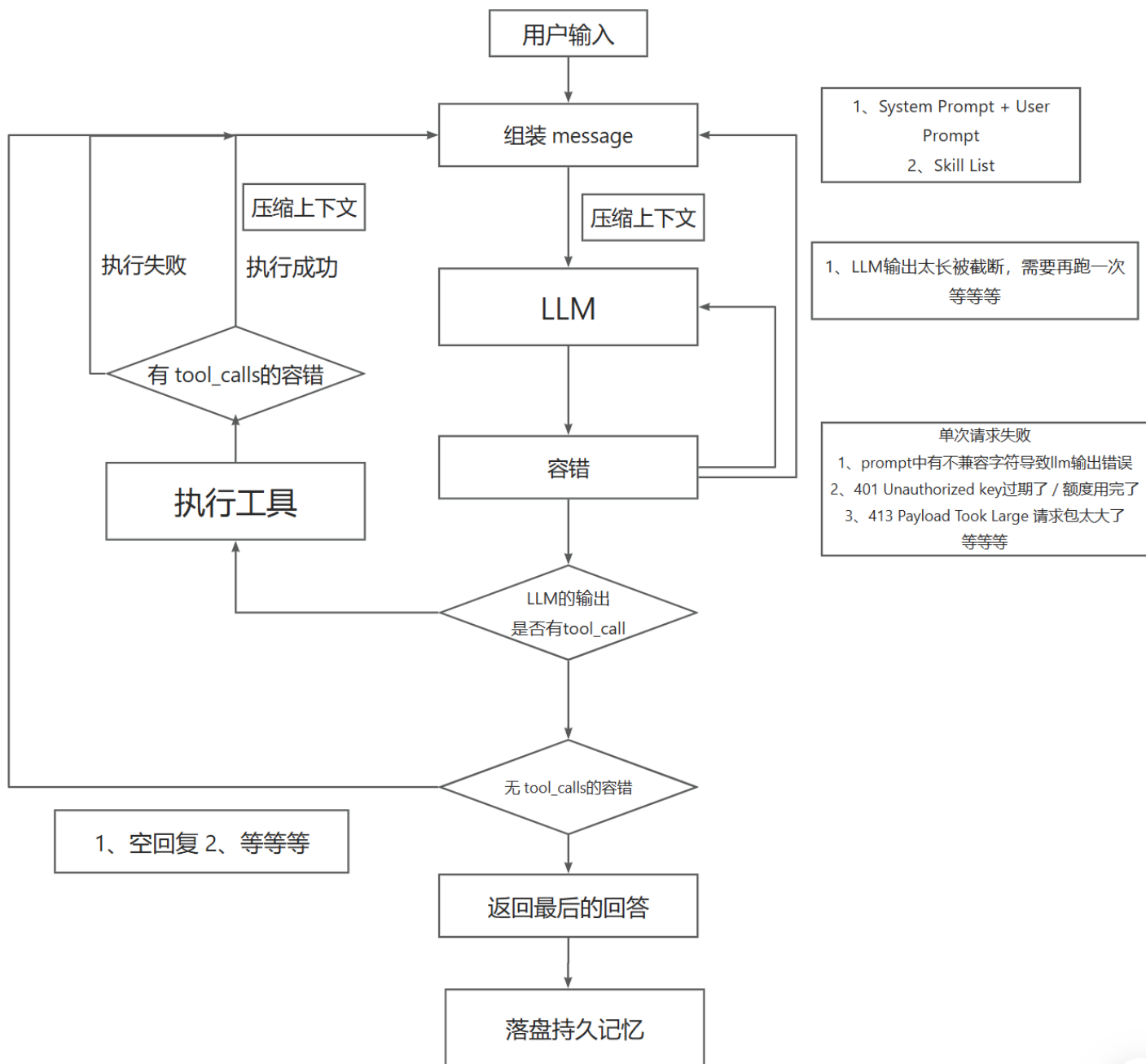
Turn: 用户发送的一次消息

跨Session: 不同的对话框, 不同的对话线程

总结起来, 从最小心智模型的简单一次turn到引入了session的概念。而最小心智模型关注的是一次turn。

而一次turn = 输入 + (模型 -> 工具 -> 模型 -> 工具 -> 模型 ->) + 输出

上下文压缩



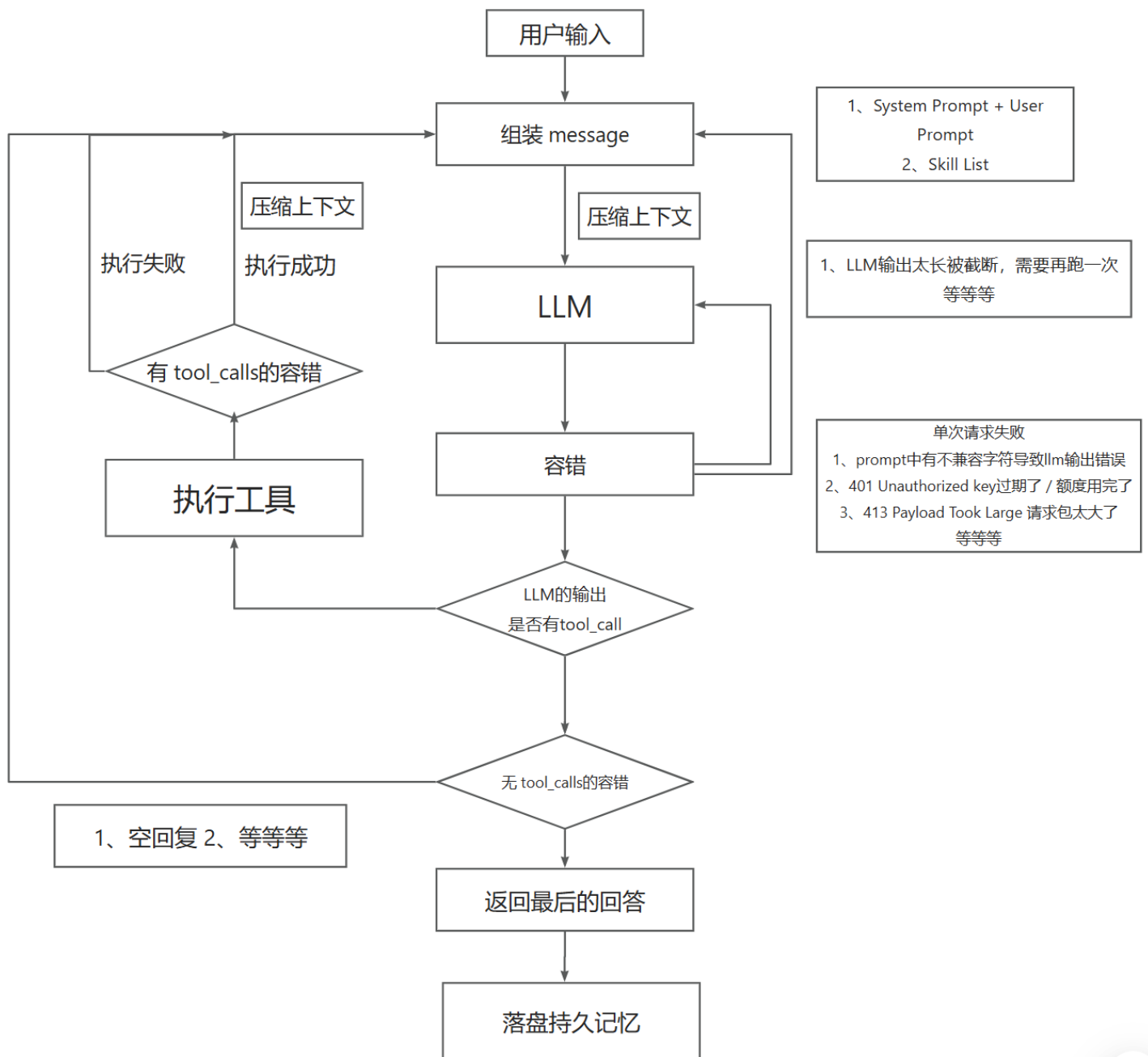
相同点：上下文窗口有限，要在不改变任务方向的前提下，把太久、太大的对话压进更小表示，再继续多轮推理与工具调用。在每次传递给模型前，检测此时的上下文长度，如果超过一定的阈值，则会进行一个压缩。

不同点：

Hermes：可以在压缩前进行一个 memory flush，保存之前的所有内容。

Claude Code：可以手动压缩。

记忆模块



考虑不同层级的记忆：

- 用户层面：不同用户有不同的记忆？单机多人的话常用 不同 Profile 或不同系统用户。
- 项目层面：不同的项目有不同的记忆？
- Seesion层面：一个项目中有不同session，不同的session层面有不同的记忆？同时，如何做到 session之间的跨记忆？

记忆：用户记忆 + 项目记忆 + 长期记忆、跨session记忆+ 同一个session的历史上下文 + 用户本次的输入 + Hook机制。

最终记忆是要作为prompt输入给模型的。

- Hermes
 - 用户记忆：通过Profile机制，每个profile有自己独立的 Hermes home 地址，所以天然隔离
 - 一套 profile 往往只有一份 MEMORY.md、USER.md

- 项目记忆
 - HERMES.md / AGENTS.md / CLAUDE.md 等。作用是：这个仓库是什么？该怎么干？等问题。
- 长期记忆、跨session记忆：人工策划的短文本事实（偏好、约定、用户画像）MEMORY.md（agent 笔记）、USER.md + Memory Provider 插件（例如：Mem0）。
 - MEMORY.md / USER.md。在会话开始构建 system prompt 时注入冻结快照。会话中可用 memory 工具写盘，但不强制立即改本会话已缓存的 system 主体。
 - 在跨会话的时候，有两个处理方法。一个是SQLite + session_search；一个是Memory Provider插件。



SQLite + session_search 与 Memory Provider的区别？

SQLite：存的是会话 transcript：谁在什么时候说了什么，偏原始对话、事件流。偏 episodic 情节记忆

Memory Provider：存的是被整理过的记忆：偏好、事实、关系、画像，或插件里的向量/图谱等，偏摘要后的知识，不是整段聊天原文。压缩后的、可维护的知识。

- 会话记忆：messages = 初始 history + 本轮 user + 本回合内产生的多轮 assistant/tool/。
 - Claude Code
 - 项目规划层：CLAUDE.md（含层级继承：全局/项目/子目录）。
 - 长期事实层：Auto memory（项目级目录）+ MEMORY.md（像入口/索引，受加载上限约束）。不是“把所有长期记忆全量塞进每轮”。

Auto memory：Claude Code会根据对话中的信息，自动判断哪些值得长期保留，然后写入到它的 memory 中。

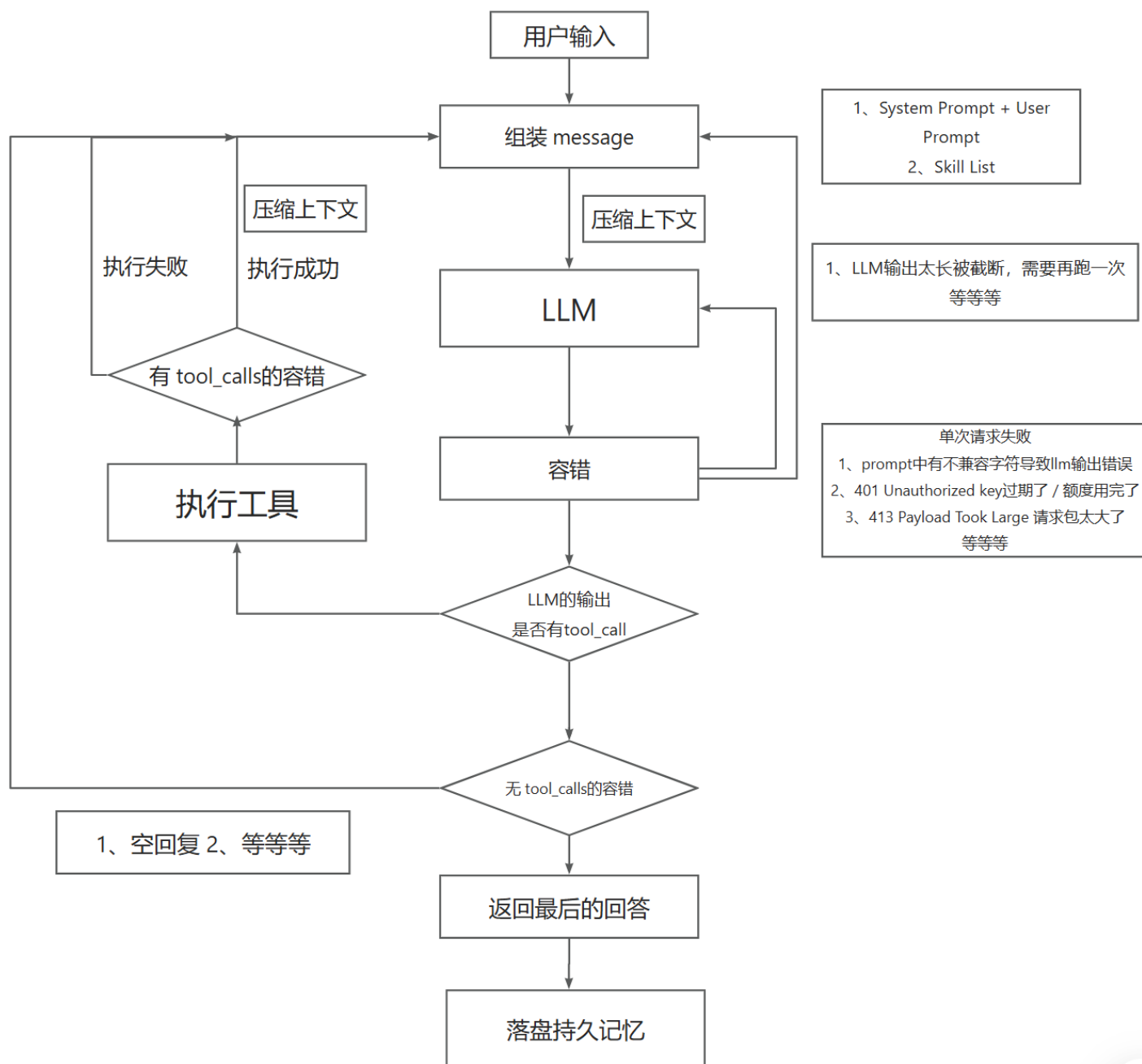
触发：回合后/会话进行中由系统自动判定。你明确说“记住这个”；你反复纠正同一偏好/约定（例如测试命令、代码风格）等。并不是每一轮都会做。
 - 会话层：当前对话完整会话上下文（直到 compact）。
- 对话中，就会持续的维护一个会话笔记。当token数达到了阈值，则会将当前的会话上下文更换成这个会话笔记。
- AutoDream：离线记忆整理。距离上次 $\geq 24h$ 且 ≥ 5 个新 session。后台fork子agent，读多个会话的信息，然后做一个总结归纳，存入到MEMORY.md文件中。

TODO / 规划层

- 概念：把大而模糊的目标拆成可执行步骤，并在执行过程中维护任务状态（进行中/完成/取消）。todo 不是推理必需，但能减轻遗忘子任务、方便用户看见进度，属于可选的规划脚手架。

- Hermes：有专门的 todo 的数据结构，存储当前的任务的todo状态。还可以从历史中恢复。
- Claude Code：有计划 / 分步工作流，但是不是暴露给用户的全局 todo 数据结构，而是内部自发的交互范式，在长任务模型中，它会自发的列出步骤，靠对话里的说明。而没有一个专门的todo工具。

Skill



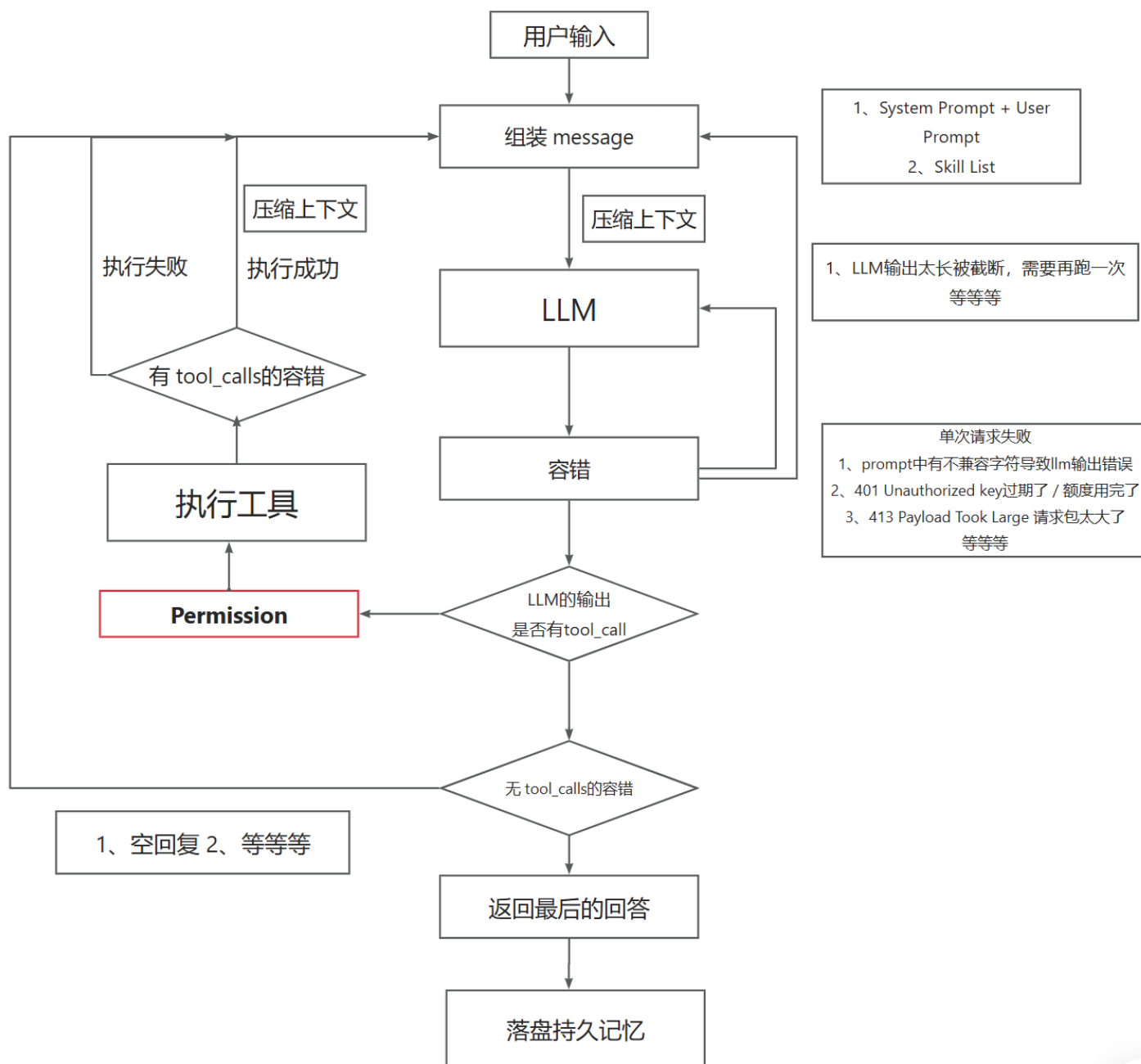
- 概念：Skill = 封装好的可复用知识与流程

机制上通常是：目录里有一批 skill，每个有元数据（名称、描述、触发条件）；运行时加载skill目录 → 按需查看正文 → 按步骤执行。

即渐进式披露、skill信息组装进 prompt

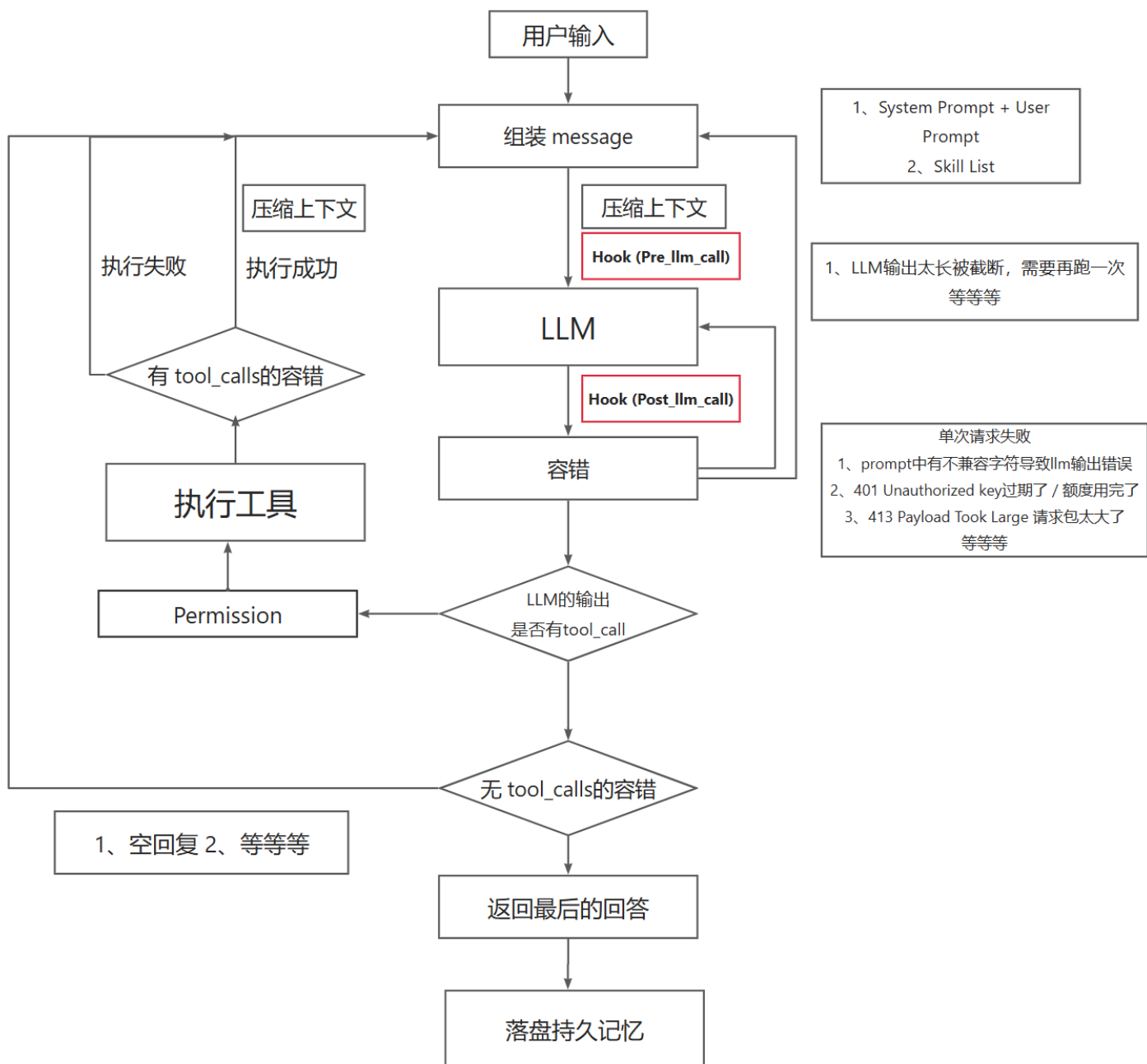
- Hermes：自进化 skill，在 System Prompt中写入了可修改skill的指引（完成复杂任务（文档里举例：5+ 次工具调用）、啃掉难搞的报错、或跑通非平凡工作流后，应把做法 `skill_manage` 存成 skill）。在完成任务的过程中发现可复用的做法，调用 `skill_manage` 工具，去创建 / 修改 skill
- Claude Code：用户维护skill，本身不修改创建skill

安全 / Permission



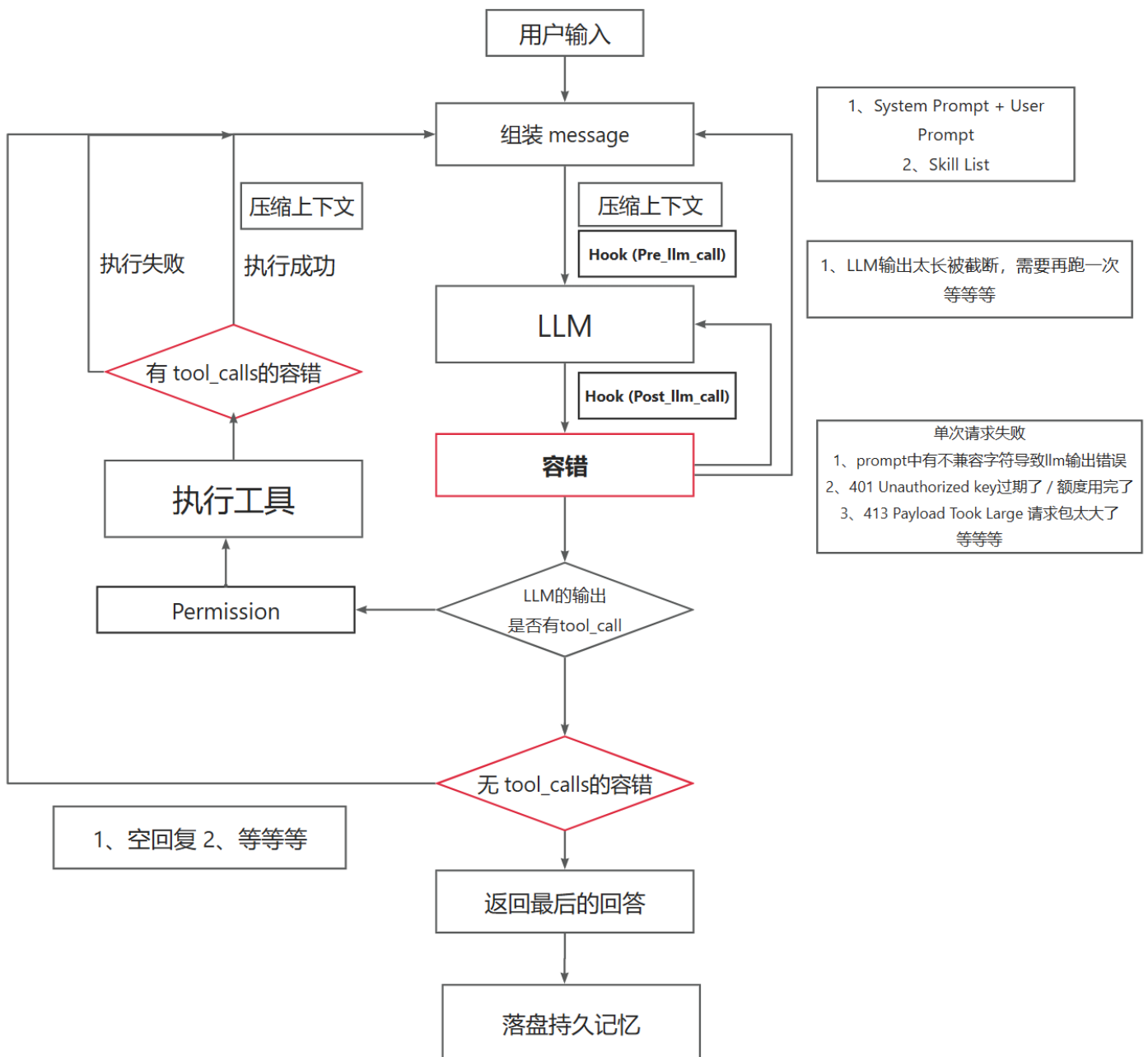
- 限制 agent 能做什么：哪些工具可用、哪些命令需批准、是否在沙箱/容器里执行、是否允许访问密钥路径等。通常就是在解析模型的工具调用时，进行一个判断

Hook机制



- 在不改核心 agent 循环的前提下，在固定生命周期点插入逻辑（前后置处理、附加上下文、观测遥测）。Hook = 扩展点；插件 = 实现这些扩展点的模块。
 - pre_llm_call、post_llm_call、pre_api_request等
 作用：在每轮进入主工具循环、真正打 LLM 之前触发。插件可以返回一段 context，拼进当前用户消息，例如：用来临时塞内部工单号、本次请求的 metadata 等。

Recovery

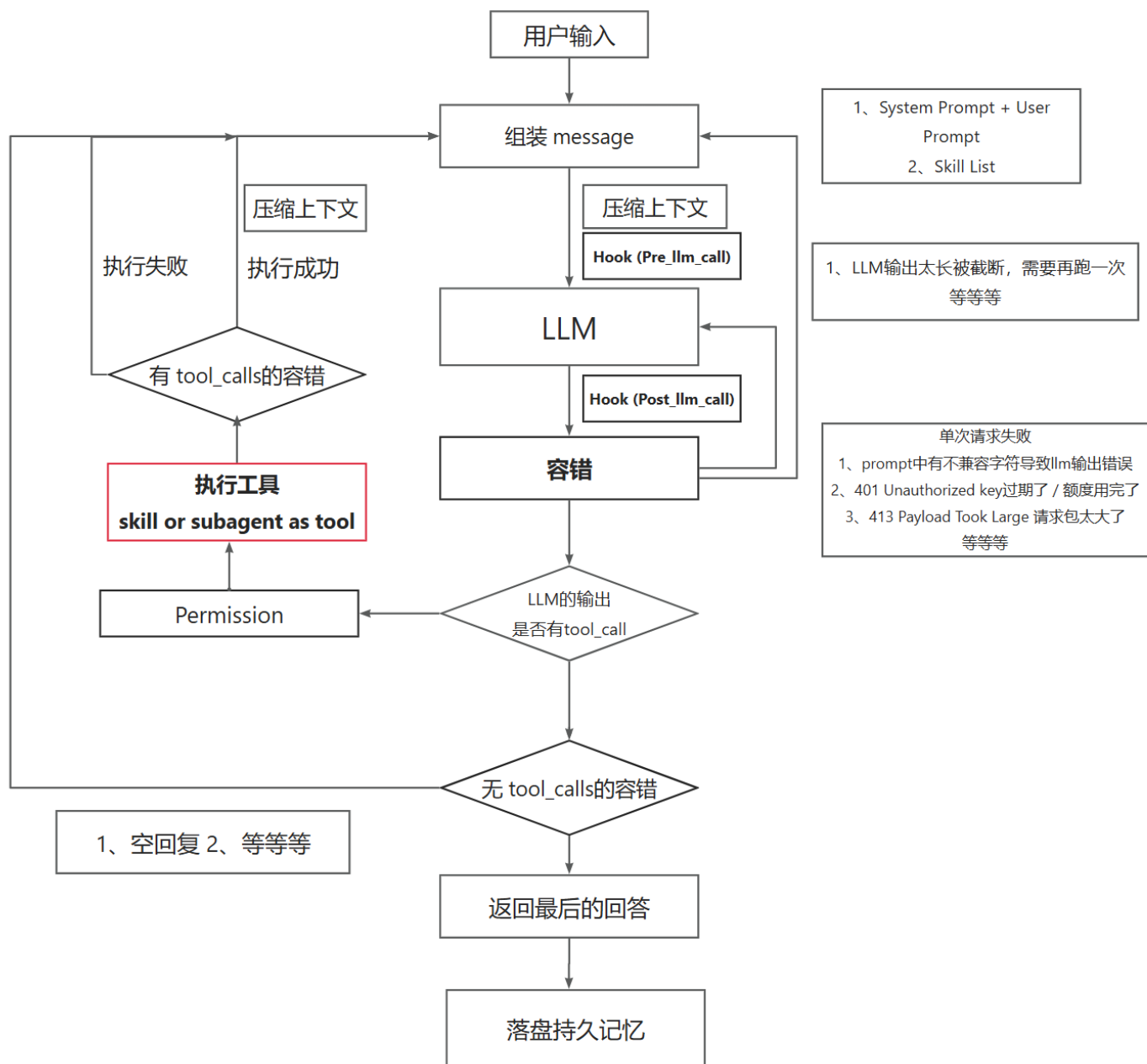


- 模型与基础设施会失败：网络、429、空响应、截断、错误工具名。分类错误 → 退避重试 → 换备用模型/凭证 → 压缩再继续，避免一轮挂死。
 - LLM单轮容错：

网络限流、429，连接断开、超时等，就是通过再跑一次LLM的调用。如果实在不行，可以走 fallback。

拿回的结果不正常，空包、格式不对，返回的内容字段类型怪异等，都会尝试重新来一次
 - 调用工具前：工具名不对、参数不合法等
 - 工具执行后：执行失败，并行或者串行执行时出错等
 - 没有工具调用：只有思考内容没有正文，模型在工具之后返回空、只有截断的思考块
- Hermes：有 fallback 的概念，可以手动配置备用模型

Subagent



- 把子任务交给独立上下文窗口执行，避免主对话被工具输出撑爆，并可并行。子 agent = 隔离的执行单元。

实现方法：将子agent也看作是一个工具，进行正常的工具调用。